

Tracking

The **Object Tracking Node** tracks detected objects, traffic signs, and intersection lane markings over time, providing stable track IDs and velocity estimates to the planner.

Internally, two or three independent Kalman filter-based trackers run in parallel (SORT architecture: Predict → Associate → Update → Create/Delete). Incoming detections are associated with existing tracks using **Mahalanobis distance** gating and the Hungarian algorithm. All confirmed tracks are bundled into a single state topic. The crossing tracker can be disabled via `crossing_tracking_enabled: false`.

Requirements

- Ros 2 Jazzy installation according to [Smartrollerz .dotfiles](#)
- [Smarty utils](#)
- [Camera Preprocessing](#)
- [Object detection](#)
- Google Coral TPU (never directly used in Tracking but required for detection), can be found in the SmartRollerz lab
- PyCoral (Run the [install_pycoral.sh](#) script that is provided by the object detection)

NOTE: There're no further Python packages required. All packages that are used are already required by Smarty utils or object detection

Running

1. Rosbags are recordings of test drives with the vehicle. They are used for testing on your Laptop and can be found on the [NAS](#). This is how you play the recording:

```
$ ros2 bag play [name of your bag]
```

NOTE: Please check the [official Ros 2 Jazzy documentation](#) for further options.

2. Run the camera_preprocessing node:

```
$ ros2 launch camera_preprocessing camera_preprocessing.launch.py
```

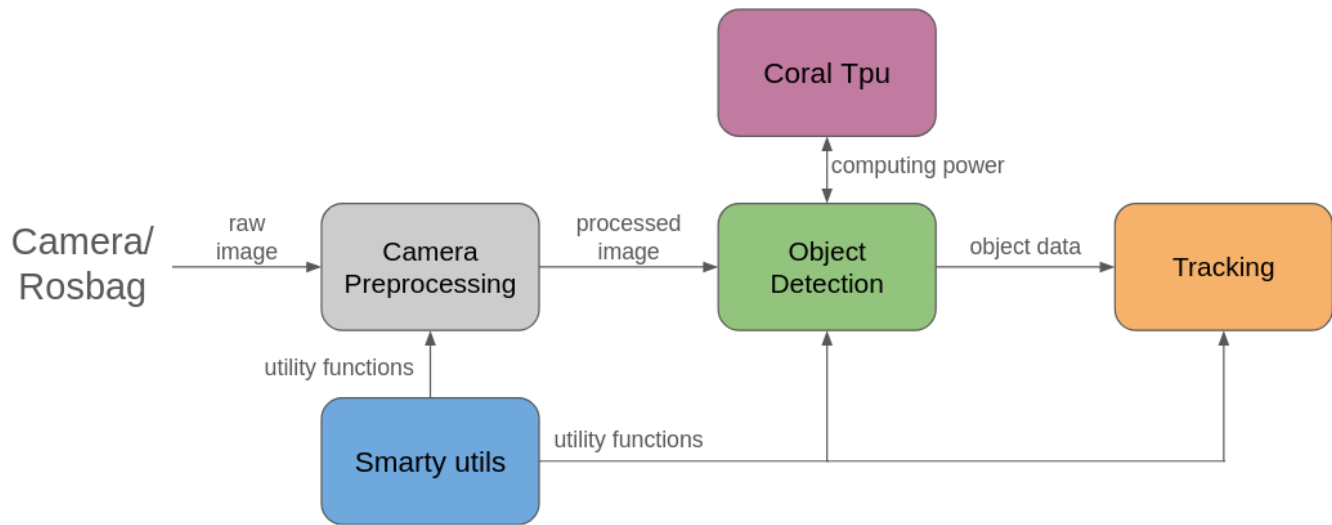
3. Run the object_detection node:

```
$ ros2 launch object_detection object_detection.launch.py
```

4. Run your tracking node:

```
$ ros2 launch tracking object_tracking.launch.py
```

This image might give you a better understanding how the tracking is set in the ecosystem.



In case you want to see a visual representation of the tracking output, you can run the visualization node:

```
$ ros2 run tracking tracking_visualization_node
```

Just like the visual outputs of the camera preprocessing and the object detection, it can be viewed in rviz:

```
$ ros2 run rviz2 rviz2
```

Interfaces (Input & Output)

1. Input (Subscribed Topics)

Topic	Message Type	Source
/object_detection/object	Float32MultiArray	Moving objects (vehicles, pedestrians)
/object_detection/sign	Float32MultiArray	Traffic signs
/crossing_detection/result	Float32MultiArray	Intersection lane markings (only if crossing_tracking_enabled: true)

Detection data structure (flattened array, 6 values per detection):

```
[Class_ID, BL_x, BL_y, BR_x, BR_y, Score]
```

Field	Description
Class_ID	Object class (int)
BL_x / BL_y	Bottom-left coordinate (mm, vehicle frame)
BR_x / BR_y	Bottom-right coordinate (mm, vehicle frame)
Score	Detection confidence (0.0 – 1.0)

2. Output (Published Topic)

Topic	Message Type
/tracking/state	state_msgs/State

The message contains an array of **TrackedObject** entries (all three trackers bundled):

Field	Type	Description
tracked_id	uint32	Stable track ID over time
class_id	uint8	Object class
position_x	float32	x-position (mm, vehicle frame)
position_y	float32	y-position (mm, vehicle frame)
velocity_x	float32	Velocity in x (mm/s)
velocity_y	float32	Velocity in y (mm/s)
confidence	float32	Tracking confidence (0.0 – 1.0)
width	float32	Object width (mm)

3. Track ID Ranges and Object Classes

Each object has 2 IDs: The **Class ID** identifies the class an object belongs to (e.g. car, pedestrian) while the **Track ID** is distinct for each object that is tracked and is used to recognize known objects. Track IDs are partitioned by tracker:

Range	Tracker	Input Topic
0 – 9999	Object Tracker	/object_detection/object
10000 – 19999	Sign Tracker	/object_detection/sign
20000+	Crossing Tracker	/crossing_detection/result

Class IDs represent the following objects and signs:

Object Tracker

Class ID	Label
2	Vehicle
10	Pedestrian

Sign Tracker

Class ID	Label
1	Stop sign
3	No overtaking
4	No overtaking lifted
5	Fast track
6	Fast track lifted
7	Speed limit 30
8	Speed limit 30 lifted
9	Crosswalk
13	Priority for oncoming traffic
14	Parking
15	Turn left
16	Turn right
17	Give way
18	Priority road
19	Pedestrian island

Crossing Tracker

Class ID	Label
20	Ego lane — solid
21	Ego lane — dotted
22	Opposing lane — solid
23	Opposing lane — dotted
24	Right lane — solid
25	Right lane — dotted
26	Left lane — solid

Class ID	Label
27	Left lane — dotted

Parameters

All parameters are configured in `config/tracking_params.yaml`.

General Parameters

Parameter	Default	Description
<code>publish_interval_ms</code>	15	State publisher frequency (ms)
<code>tracker_step_interval_ms</code>	40	Minimum interval (ms) for a timer-triggered predict step when no detection arrives
<code>max_x</code>	3000	Maximum valid x-position (mm) — tracks outside this range are deleted
<code>max_y</code>	2000	Maximum valid y-position (mm)
<code>crossing_tracking_enabled</code>	true	Enable/disable the crossing tracker; set to <code>false</code> to skip crossing detection entirely

Per-Tracker Parameters (prefix: `object_` / `sign_` / `crossing_`)

Parameter	Description
<code>max_age</code>	Maximum tracker frames without a measurement update before a track is deleted
<code>min_hits</code>	Minimum number of successful measurement updates required to confirm a track
<code>min_age</code>	Minimum number of tracker frames required to confirm a track
<code>max_distance</code>	Mahalanobis distance gate for detection-to-track association (χ^2 distance)
<code>q_pos</code>	Process noise for position (mm) — higher = model is trusted less, predictions spread faster
<code>q_vel</code>	Process noise for velocity (mm/s) — higher = larger velocity changes are expected
<code>r_pos</code>	Measurement noise for position (mm) at the reference distance — higher = measurements are trusted less
<code>r_dist_ref</code>	Reference distance (mm) for measurement noise scaling — <code>r_pos</code> applies exactly at this distance, scaling linearly with distance
<code>sigma_pos_init</code>	Initial position uncertainty when creating a new track (mm)
<code>sigma_vel_init</code>	Initial velocity uncertainty when creating a new track (mm/s)

Note on `max_distance`: A value of 3.03 corresponds to the χ^2 99% confidence interval (2 DoF). Higher values allow more generous association and reduce track fragmentation, but increase the risk

of incorrect matches.

Coordinate System

- **Units:** All positions and velocities in **millimeters (mm)** and **mm/s**
 - **Origin:** Vehicle-relative frame (as received from the detection node)
 - **No ego-motion compensation:** Since no ego-velocity is available, signs appear to move in the vehicle frame when the vehicle drives past them
-

Debugging

```
# Check tracker output (are IDs stable? do positions change plausibly?)
ros2 topic echo /tracking/state

# Check whether input is arriving
ros2 topic echo /object_detection/object
ros2 topic echo /object_detection/sign
ros2 topic echo /crossing_detection/result
```

For more detailed logging, set `debug: true` in `tracking_params.yaml`.

For performance analysis, set `export_timing_csv: true` — on node shutdown, a CSV with per-frame timing data is written to `performance_measurements/`. The scripts in the `/test` folder can be used to visualize and compare this data.

Performance & Evaluation

Timing Analysis

Set `export_timing_csv: true` in `tracking_params.yaml`, then run the node normally (e.g. via rosbag replay). On shutdown a CSV is written to `performance_measurements/`.

Visualize a single recording:

```
python3 test/plot_timing.py
performance_measurements/tracking_timing_<date>.csv

# Wall-clock time on x-axis instead of frame number:
python3 test/plot_timing.py tracking_timing.csv --time
```

Generates per-tracker line charts and stacked area charts in a folder next to the CSV.

Compare two recordings (e.g. before/after a parameter change):

```
python3 test/compare_timing.py before.csv after.csv
python3 test/compare_timing.py before.csv after.csv --labels "Before"
"After" --time
```

Outputs overlaid line charts and a side-by-side statistics table.

Detection vs. Tracking Evaluation

`tracking/evaluate_node.py` runs against a rosbag and ground-truth annotations and writes one JSON result file per pipeline stage (detection, tracking):

```
ros2 run tracking evaluate_node --mode detection --gt ground_truth.csv --
out eval_detection.json
ros2 run tracking evaluate_node --mode tracking --gt ground_truth.csv --
out eval_tracking.json
```

Visualize results (F1-Score and mean localisation error):

```
python3 test/plot_evaluation.py eval_detection.json eval_tracking.json
python3 test/plot_evaluation.py eval_detection.json eval_tracking.json --
out plots/ --labels "Detection" "Tracking"
```

Generates a bar chart comparing both pipeline stages per class.